



16

React Native under the Hood

The previous chapter briefly touched on what React Native is and the differences that users experience between the React Native UI and mobile browsers.

In this chapter, we will dig deeper into React Native, becoming well-versed on how it performs on mobile devices and what we should attain before commencing any efforts with this framework. We will also look at what options we can execute for the native functionality of JavaScript and what restrictions we will be up against.

We will cover the following topics:

- Exploring the React Native architecture
- Explaining JavaScript and Native modules
- Exploring React Native components and APIs

Exploring the React Native architecture

Before understanding how React Native works, let's revisit some historical points about the React architecture and the differences between web and native mobile apps.

The state of web and mobile apps in the past

Meta released **React** in 2013 as a monolith tool for creating apps, using a component approach and a **virtual DOM**. It gave us the opportunity to develop web applications without thinking about browser processes, such as parsing JS code, creating the DOM, and handling layers and rendering. We just had to create interfaces using state and props for data and CSS for styling, fetch data from the backend, save it in local storage, and so on.

React, together with browsers, allowed us to create a performance application in less time. At that time, the architecture of React looked like this:

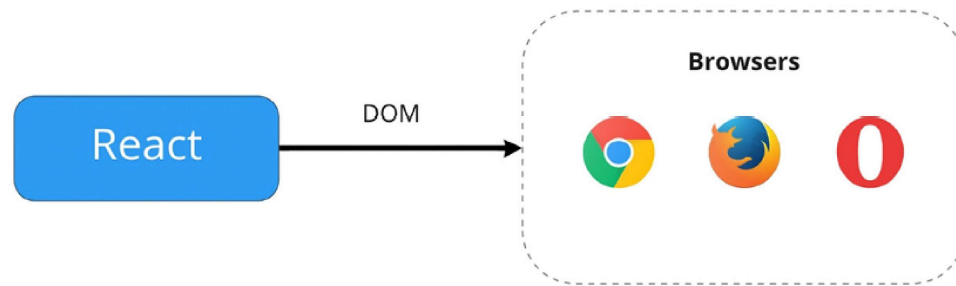


Figure 16.1: React architecture in 2013

The new declarative approach to developing interfaces became more favorable because of the fast development and the low threshold for novices. Additionally, if your backend is built with **Node.js**, you can benefit from the ease of support and development of the entire project using just one programming language.

At the same time, mobile apps require more complex techniques to create apps. For Android and iOS apps, companies should manage three different teams with unparalleled experience to support three major ecosystems:

- Web developers should know HTML, CSS, JS, and React.
- **Java** or **Kotlin** SDK experience is required for Android developers.
- The iOS developer should be familiar with **Objective-C** or **Swift** and **CocoaPods**.

Every step of developing an application, from prototyping to release, requires unique skills. Web and mobile app development before cross-platform solutions looked like this:



Figure 16.2: The state of web and mobile apps

Even if a corporation carries out a basic application, it can be faced with some major issues:

- Each of these teams implements the same business logic.
- There is no alternative to sharing code between teams.
- It is not conceivable to share resources between teams (Android developers can't write code for iOS applications, and vice versa).

As a result of these significant issues, we likewise have complications with having more testing resources, since there are more places to create bugs. The speed of development is also diverse because mobile apps take more time to deliver the same features. This all accumulates into a large, costly problem for the companies involved. Many of them came up with ideas

on how to write a single code base or reuse a current one that can be used in multiple ecosystems. The simplest method would be to wrap a web app for mobile using a browser, but this has limitations in handling touch and gestures, as we explored in *Chapter 15, Why React Native?*.

In response to these issues, Meta started investing resources in developing a cross-platform framework and released the **React Native library** in 2015. Also, it divided React into two separate libraries. To render our app in the browser, we should now use the **ReactDOM** library.

In *Figure 16.3*, we can see how **React** works together with **ReactDOM** and **React Native** to render our apps:

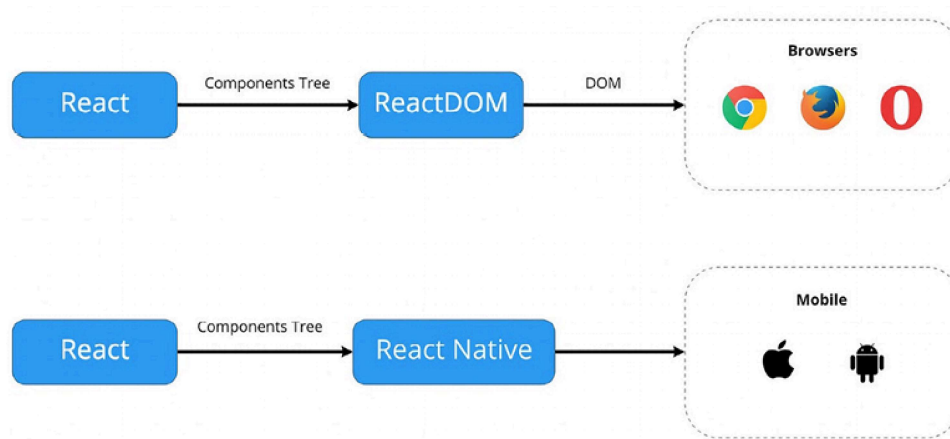


Figure 16.3: ReactDOM and React Native flow

Now, React only works to manage the components tree. This approach encapsulates any rendering APIs and hides a lot of platform-specific methods from us. We can concentrate solely on developing interfaces and cease speculating about how they would be rendered.

That's why React is frequently claimed as a renderer-agnostic library. Also, for web apps, we use ReactDOM, which forms elements and applies them right to the browser DOM. For mobile apps, React Native renders our interface directly on the mobile screen.

But how does React Native replace the whole browser API and allow us to write familiar code and run it on mobiles?

React Native current architecture

The React Native library allows you to create native applications with React and JS by utilizing native building blocks. For instance, the `<Image/>` component represents two other native components, `ImageView` on Android and `UIImageView` on iOS. This is viable because of the architecture of React Native, which includes two dedicated layers, represented by **JS** and **Native** threads:

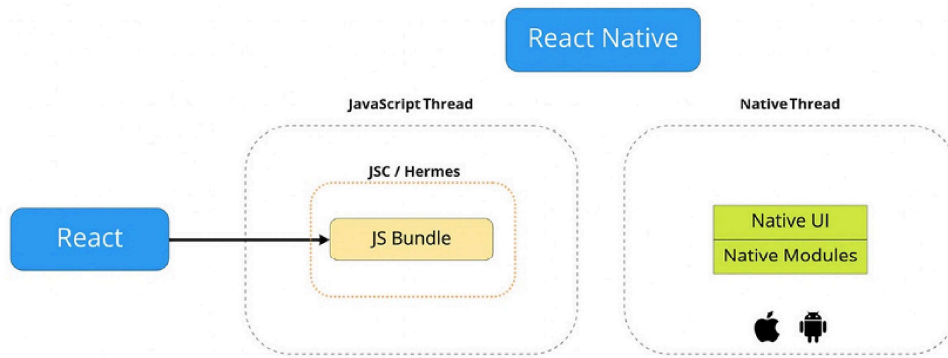


Figure 16.4: React Native threads

In the next sections, we will explore each thread and see how they can communicate, ensuring that JS is integrated into the native code.

JS as part of React Native

As the browser executes JS through **JS engines** such as **V8**, **SpiderMonkey**, and others, React Native also contains a **JS virtual machine**. There, our JS code is executed, API calls are made, touch events are processed, and many other processes occur.

Initially, React Native only supported Apple's **JavaScriptCore** virtual machine. With iOS devices, this virtual machine is built-in and available out of the box. In the case of Android,

JavaScriptCore is bundled with React Native. This increases the size of the app.

Therefore, the *Hello World* application of React Native would consume approximately 3 to 4 MB on Android. From the 0.60 version, React Native started using the new **Hermes virtual machine**, and from 0.64, provided support for iOS as well.

The Hermes virtual machine introduced a lot of improvements for both platforms:

- Improvement of the app's startup time
- A size reduction of the downloaded app
- Decreased memory usage
- Built-in proxy support, enabling the use of **react-native-firebase** and **mobx**

Understanding the comparative benefits between the old and new architectures is a relatively common topic in interviews. More information about Hermes can be found here:

<https://reactnative.dev/docs/hermes>.

JS in React Native, as in browsers, is implemented in a single thread. That thread is responsible for executing JS. The business logic we write is carried out in this thread. This means all our common code, such as components, state, Hooks, and REST API calls, will be handled in the JS part of the app.

Our entire application structure is packaged into a single file using the **Metro** bundler. It is also responsible for transpiling JSX code into JS. If we want to use TypeScript, **Babel** can support it. It works right out of the box, so there's no need to configure anything. In future chapters, we will learn how to start a ready-to-work project.

The “Native” part

Here is where native code is executed. React Native implements this part in native code for each platform: Java for Android and Objective-C for iOS. The **Native** layer is mainly composed of Native modules that communicate with the Android or iOS SDK and are supposed to provide native functionality for our apps, using a unified API. If we want to display an alert dialog, for instance, the **Native** layer presents a unified API for both platforms, which we will call from the JS thread using the single API.

This thread interacts with the JS thread when you need to update the interface or call the native functions. There are two parts to this thread:

- The first, the **React Native UI**, is responsible for using native interface shaping tools.
- The second is **Native Modules**, which allow applications to access the specific capabilities of the platform on which

they run.

Communication between threads

As previously mentioned, each React Native layer implements a unique API for every native and UI feature in an application. The communication between layers is accomplished through the bridge. The module is written in C++ and is based on an asynchronous queue. When the bridge receives data from one of the parties, it serializes it, converts it to a **JSON** string, and passes it through the queue. After arriving at its destination, the data is deserialized.

As shown in the alert example, the native part accepts the call from JS and displays the dialog. In reality, the JS method, upon being invoked, sends a message to the **bridge**, and upon receiving this message, the Native part executes the instruction. Native messages may also be forwarded to the **JS** layer. On clicking the button, for example, the **Native** layer sends a message to the **JS** one with an `onClick` event. It can be imagined as follows:

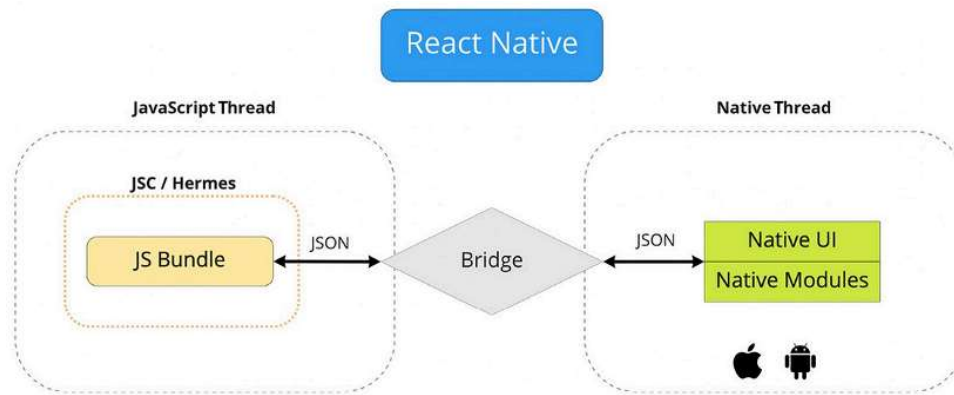


Figure 16.5: The bridge

JS and the Native part of this architecture, together with the bridge, resemble the server and client sides of web applications, where they communicate through the REST APIs. It does not matter to us in which language or how the Native part is implemented, since the code in JS is isolated. We simply send messages and receive responses from the bridge. This is both a significant advantage and a great disadvantage: first, it allows us to implement cross-platform apps with one code base, but it can be a bottleneck in our app when we have a lot of business logic in it. All events and actions in the application rely on asynchronous JSON-bridged messages. Each party sends these messages, expecting that sometime in the future, a response will be received from these messages (which is not guaranteed). With such a data exchange scheme, there is a risk of overloading the communication channel.

Here is an example commonly used to illustrate how such a communication scheme can cause performance problems for an application. Suppose a user of an application scrolls through a huge list. When the `onScroll` event occurs in the native environment, information is passed asynchronously to the JS environment. But native mechanisms do not wait until the JS part of the application does its job and reports to them about it. Because of this, there is a delay in the appearance of empty space in the list before displaying its contents. We can avoid a lot of usual problems using special approaches, such as using paginated `FlatList` on limitless lists. We will look at the main tricks in future chapters, but it is important to remember the limitations of the current architecture.

Styling

As we already understand the cross-platform concept, we can assume that each platform has its own technologies for creating and styling interfaces. In order to unify this, React Native has a **CSS-in-JS** syntax to style an app. Using **Flexbox**, components are able to specify the layout of their children. This ensures a consistent layout across different screen sizes. It is usually similar to how CSS works on the web, except the names are written in camel case, such as `backgroundColor` rather than `background-color`.

In JS, it is a plain object with style properties, and in native code, it is a separate thread called **Shadow**. It recalculates the layout of the application using the **Yoga** engine, which was developed by Meta. In this thread, the calculations related to the formation of the application interface are performed. The results of these calculations are sent to the Native UI thread responsible for displaying the interface.

With all the parts coming together, the final architecture of React Native is illustrated in this figure:

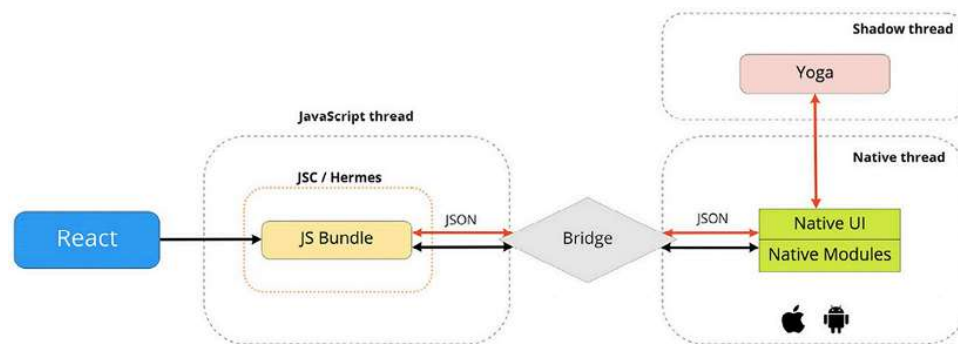


Figure 16.6: The current React Native architecture

The current architecture of React Native addresses major business problems: it is feasible to develop web and mobile applications within the same team, it is possible to reuse a large amount of business logic code, and even developers with no

previous experience in mobile development can easily use React Native.

However, the current architecture is not ideal. Over the past few years, the React Native team has been working on a bridge bottleneck solution. The new architecture is designed to address this issue.

React Native future architecture

A series of significant improvements have been introduced to React Native that will streamline the development process and make it more convenient for everyone.

React Native's re-architecture will gradually deprecate the bridge and replace it with a new component called the **JS Interface (JSI)**. In addition, this element will enable new **Fabric** components and **TurboModules**.

The use of the JSI opens up many possibilities for improvement. In *Figure 16.7*, you can see the major updates to the React Native architecture:

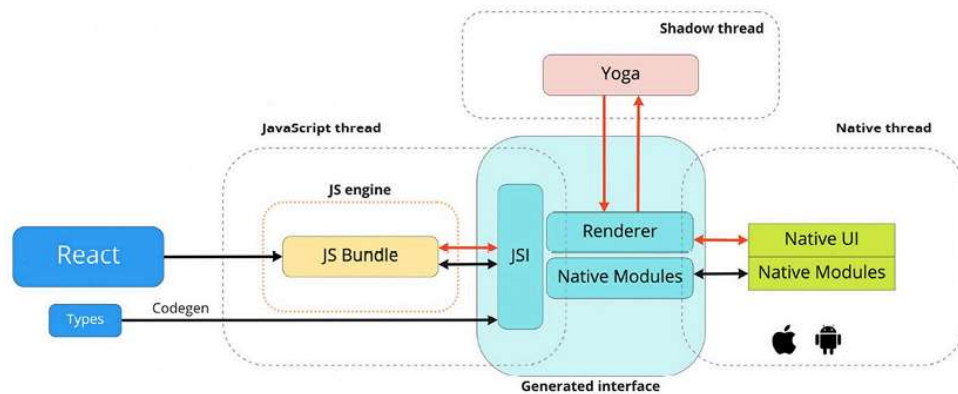


Figure 16.7: The new React Native architecture

The first change is that the JS bundle is no longer dependent on a **JavaScriptCore** virtual machine. It is actually part of the current architecture because, now, we can enable the new **Hermes JS engine** on both platforms. In other words, the JavaScriptCore engine can now easily be replaced with something else, quite possibly with better performance.

The second improvement is what lies at the heart of the new React Native architecture. The JSI allows JS to call native methods and functions directly. This was made possible by the **HostObject** C++ object, which stores references to native methods and properties. **HostObject** in JS binds native methods and props to a global object, so direct calls to JS functions will invoke Java or Objective-C APIs.

Another benefit of the new React Native is the ability to fully control native modules called **TurboModules**. Rather than starting them all at once, the application will only use them when they are needed.

Fabric is the new UI manager, called **Renderer** in *Figure 16.7*, which is expected to transform the rendering layer by eliminating the need for bridges. It is now possible to create a **Shadow Tree** directly in C++, which increases speed and reduces the number of steps to render a particular element.

In order to ensure smooth communication between React Native and Native parts, Meta is currently working on a tool called **CodeGen**. It is expected to automate the compatibility of strongly typed native code and dynamically typed JS to make them synchronize. With this upgrade, there will be no need to duplicate the code for both threads, thereby enabling smooth synchronization.

The new architecture could open the way for the development of new designs that are capable of things that were not available in old React Native applications. The fact is that we now have at our disposal the power of C++. This means that with React Native, it will now be possible to create many more varieties of applications than before.

Here, we discussed the fundamentals that explain how React Native works. It is important to understand the architecture of the tools we use. Having this knowledge allows you to avoid mistakes during planning and prototyping, as well as maximize the potential of your future applications. In the following section, we will briefly explore how to extend React Native with modules.

Explaining JS and Native modules

React Native does not cover all the native capabilities out of the box. It only provides the most common features that you will need in a basic application. Also, the Meta team itself has recently moved some functions into its own modules in an effort to reduce the size of the overall application. For example, `AsyncStorage`, for storing data on a device, was moved into a separate package and must be installed if you plan to use it.

However, React Native is an extendable framework. We can add our own native modules and expose the JS API using the same bridge or JSI. Our focus in this book will not be on developing native modules, since we need prior experience with Objective-C or Java. Also, it is not necessary, since the React community has created an enormous number of ready-to-use

modules for all cases. We will learn how to install native packages in subsequent chapters.

The following are a few of the most popular native modules, without which most projects couldn't prosper.

React Navigation

React Navigation is one of the best React Native navigation libraries for creating navigation menus and screens for your app. It's a good tool for beginners because it's stable, fast, and less buggy. The documentation is really good, and it provides examples for all use cases.

We'll learn more about React Navigation in *Chapter 19, Navigating between Screens*.

UI component libraries

The UI component libraries enable you to quickly assemble an application layout without wasting time designing and coding atomic elements. In addition, such libraries are often more stable and consistent, which leads to better results both in terms of UI and UX.

These are some of the most popular libraries (we will explore a few of them in greater detail in future chapters):

- **NativeBase:** This is a component library that enables developers to build universal design systems. It is built on top of React Native, allowing you to develop apps for Android, iOS, and the web.
- **React Native Element:** This provides an all-in-one UI kit for creating apps in React Native.
- **UI Kitten:** This is a React Native implementation of the **Eva Design System**. The framework contains a set of general-purpose UI components styled in a similar way.
- **React-native-paper:** This is a collection of customizable and production-ready components for React Native, following Google's Material Design guidelines.
- **Tamagui:** This UI kit provides components that can run on mobiles and the web.

Splash screen

Adding a splash screen to your mobile app can be a tedious task, since this screen should appear before the JS thread begins. The **react-native-bootsplash** package allows you to create a fancy splash screen from the command line. The package will do all the work for you if you provide it with an image and a background color.

Icons

Icons are an integral part of the visualization of interfaces. Different approaches are used to display icons and other vector graphics on each platform. React Native unifies this for us but only with additional libraries such as **react-native-vector-icons**. Using **react-native-svg**, you can also render **scalable vector graphics (SVGs)** in a React Native app.

Handling errors

Usually, when we develop a web application, we are able to handle errors without any difficulty, since they do not reach beyond the scope of JS. As a result, we have more control and stability in the event of critical bugs because if the application does not start at all, we can easily see the reason and open the logs in **DevTools**.

There are even more complications with React Native applications, since we have a Native component in addition to the JS of the environment, which can also cause errors in application execution. Therefore, when an error occurs, our application will close immediately. It will be hard for us to figure out why.

`react-native-exception-handler` provides a simple technique for handling native and JS errors and providing feedback. To make it work, you need to install and link the module.

Then, register your global handler for JS and native exceptions, as follows:

```
import { setJSExceptionHandler, setNativeExceptionHandler }
  from "react-native-exception-handler";
setJSExceptionHandler((error, isFatal) => {
  // ...
});
const exceptionhandler = (exceptionString) => {
  // your exception handler code here
};
setNativeExceptionHandler(
  exceptionhandler,
  forceAppQuit,
  executeDefaultHandler
);
```

The `setJSExceptionHandler` and `setNativeExceptionHandler` methods are custom global error handlers. If a crash occurs, you can show an error message, use Google Analytics to track it, or use a custom API to inform the development team.

Push notifications

We live in a world where notifications are integral. We open dozens of apps every day just because we receive notifications from them.

Push notifications are often connected to a gateway provider that sends messages to users' devices. The following libraries can be used to add push notifications to your application:

- `react-native-onesignal` : A OneSignal provider for push notifications, email, and SMS
- `react-native-firebase` : Google Firebase
- `@aws-amplify/pushnotification` : AWS Amplify

Over-the-air updates

As part of a normal application update, when you build a new version and upload it to the app store, you can replace the JS package **over the air** (OTA). As the bundle contains only one file, updating it is not complicated. You can update your application as often as you like without waiting for Apple or Google to verify your application. That is the real power of React Native.

We can use it due to the **CodePush** service made by Microsoft. You can find more information about CodePush here:

<https://docs.microsoft.com/en-gb/appcenter/distribution/codepush/>.

Expo also supports OTA updates with the `expo-updates` package.

JS libraries

As for JS (non-native) modules, we have almost no restrictions, except for libraries that use unsupported APIs, such as the DOM and Node.js. We can use any packages written in JS: Moment, Lodash, Axios, Redux, MobX, and a thousand others.

We have barely scratched the surface of the possibilities to extend an application with various modules in this section. Because React Native has thousands of libraries, it makes little sense to go through them all. In order to find the required package you need, there is a project called **React Native Directory** that has collected and rated a huge list of packages. The project can be found here: <https://reactnative.directory/>.

We now know how React Native is organized internally and how we can expand its functionality. Our next step is to examine what API and components this framework offers.

Exploring React Native components and APIs

The main modules and components will be discussed in detail in each new chapter, but for now, let's familiarize ourselves

with them. A number of core components are available in the React Native framework for use in an app.

Almost all apps use at least one of these components. These are the fundamental building blocks of React Native apps:

- **View** : The main brick of any app. This is the equivalent of `<div>`, and on mobiles, it is represented as `UIView` or `android.view`. Any `<View/>` component can nest inside another `<View/>` component and can have zero or many children of any type.
- **Text** : This is a React component for displaying text. As with `View`, `<Text/>` supports nesting, styling, and touch handling.
- **Image** : This displays images from a variety of sources, such as network images, static resources, temporary local images, and images from the camera roll.
- **TextInput** : This allows users to input text using a keyboard. Props enable a variety of features that can be configured, including auto-correction, auto-capitalization, placeholder text, and different keyboard types, such as a numeric keypad.
- **ScrollView** : This component is a generic container for scrolling multiple views and components. There can be both vertical and horizontal scrolling (by adjusting the horizontal property) for the scrollable items. If you need to ren-

der a huge or limitless list of items, you should use `FlatList`. This supports a set of special props such as **Pull to Refresh** and **Scroll loading** (lazy-loading). If your list needs to be divided into sections, then there is also a special component for this: `SectionList`.

- `Button`: React Native has advanced components that can be used to create custom buttons and other touchable components, such as `TouchableHighlight`, `TouchableOpacity`, and `TouchableWithoutFeedback`.
- `Pressable`: This gives more precise touch control with React Native version 0.63. Basically, it is a wrapper for detecting touch. It is a well-defined component that can be used instead of touchable components such as `TouchableOpacity` and `Button`.
- `Switch`: This component resembles a checkbox; however, it is presented in the form of a switch, which we are familiar with on mobile devices.

In the following chapters, we will delve deeper into common components and their properties, as well as explore new components that are rarely used. We'll also look at code examples that show how to combine components to create application interfaces.

Detailed information about all the available components can be found at <https://reactnative.dev/docs/components-and->

apis.

Summary

In this chapter, we looked at the history of the cross-platform framework React Native and what problems it solved for companies. With it, companies can use a single universal developer team to build one business logic and apply it to all platforms simultaneously, thus saving a lot of time and money.

Considering, in detail, how React Native works under the hood allows us to identify potential issues at the planning stage and resolve them.

Additionally, we started to examine React Native's basic components, and with each new chapter, we will learn more about them.

In the next chapter, you'll learn how to get started with new React Native projects.